

SAVANNAH INFORMATICS / ENGINEERING

WEB ENGINEER TAKE-HOME ASSESSMENT

Any modern frontend framework · TypeScript encouraged

Allow 3–5 days · 6–8 hours effort

AI tools: permitted, declared and defended

4 SECTIONS

All connected

SINGLE SCENARIO

Clinic stock console

CLOUD DEPLOYMENT

Provider of your choice

CI/CD REQUIRED

Auto-deploy on merge

All four sections are built around a single scenario. What you design in Section 1 is what you build in Section 2 and deploy in Section 3. A 75-minute live session follows your submission.

INSTRUCTIONS

What to submit

1. A link to your GitHub or GitLab repository, public or shared with us.
2. A link to your deployed, running application.
3. A README covering your design decisions, how to run it locally, and a short note on your CI/CD setup.
4. Your Section 4 reflection, in the README or as a separate file.

Rules

- AI tools are permitted and expected, within the limits set out below. Read that section before you start.
- Use any framework, or none. Angular, React and Vue are all fine.
- You may use a component library. If you do, say which parts you configured yourself.
- Where the requirements are ambiguous, make a decision and record it in your README. Some of the ambiguity is deliberate.
- Clear reasoning matters as much as working code. A well argued README next to a working app carries real weight.

USING AI ON THIS ASSESSMENT

We use AI tools on this team every day and we expect you to use them here. We also need this assessment to tell us about your engineering, not your model's. So the rule is not abstinence. It is that you direct the tool, you declare where you used it, and you can defend everything you submit.

Use it freely for

- Project scaffolding, build configuration, and tooling setup.
- Boilerplate components, repetitive markup, and type definitions.
- Test scaffolding, once you have decided what is worth testing.
- Drafting and tidying documentation.
- Exploring an unfamiliar API, or comparing two approaches you are weighing.

Please do not delegate

- **Your Section 1 design and decision log.** Write the first draft yourself, in your own words, before you generate any implementation. Using AI afterwards to pressure-test that design is fine, and worth telling us about.
- **Your Section 4 reflection.** It is about your process. A generated answer is obvious and scores nothing.
- **Any decision you cannot explain.** If a tool proposes an approach you do not understand, either take the time to understand it or choose one you do.

What we require of you

- **Declare it.** In your README, note per section what you used AI for. A few lines is enough.
- **Own it.** In the live session we will ask you to explain any part of your code, and what breaks if we remove it. Code you cannot explain is treated as not yours and scored accordingly.
- **Report your real time spent.** Do not round it down. We are calibrating this assessment and honest numbers help us.

NOTE: We are not testing whether you can work without AI. We are testing whether you are steering it. A smaller submission you fully understand will score higher than a larger one you cannot defend.

What we are actually measuring

Anyone can generate a product listing page now. We are looking at what happens around the edges: what your app does when the network is slow, when a request fails, when the user hits refresh, when they use a keyboard instead of a mouse. That is where the difference between engineers shows up, so that is where we look.

THE DATA SOURCE

Build against DummyJSON, a free public REST API. No signup or key required. Docs: <https://dummyjson.com/docs>

The endpoints you will need

Purpose	Endpoint
Sign in	POST /auth/login with { username, password, expiresInMins }
Current user	GET /auth/me with Authorization: Bearer <accessToken>
Refresh token	POST /auth/refresh with { refreshToken }
List items	GET /products?limit=&skip=&sortBy=&order=&select=
Search items	GET /products/search?q=
Categories	GET /products/categories
Items in a category	GET /products/category/{slug}
Single item	GET /products/{id}
Update an item	PUT /products/{id}

Test credentials: any user from <https://dummyjson.com/users>. For example emilys / emilyspass.

Two things the API gives you that you should use while building

- Add ?delay=2000 to any request to slow it down. Delay accepts 0 to 5000 milliseconds.
- GET /http/{code} returns that HTTP status, so <https://dummyjson.com/http/500> gives you a real 500 to handle.

BEFORE YOU START: Read the docs properly before you start. There are behaviours in there that will affect your design.

The scenario

"We run a clinic. Our supplies team needs an internal console to see what stock we hold. They need to search it, filter it by category, sort it, open an item to see the detail, and correct the stock count when a physical count disagrees with the system. Most of them are on ward tablets over patchy wifi. Some of them share links to specific items over chat. We are starting with one clinic but this will roll out to more."

DummyJSON returns generic retail products. Treat the product catalogue as the clinic's stock catalogue. Do not spend time inventing clinical content that the data does not contain.

What to document

Before writing code, write your design into the README. Cover:

1. The components you identified and how you divided the screen up.
2. Where each piece of state lives, and why there rather than somewhere else. Server data, URL state and local UI state are not the same thing.
3. How you fetch, cache and invalidate data.
4. Your approach to layout, spacing, colour and typography. If you are using design tokens or a theme, say so. If you are using a library's defaults, say that instead.
5. Your accessibility approach.

Decisions we want to see argued

Include a decision log with at least three entries. Each entry names the decision, the alternative you rejected, and why. Generic entries score nothing. We are looking for the decisions that were actually hard on this brief.

Build the console. Deploy it in Section 3.

Required behaviour

Sign in. Users sign in before they can see stock. Request a short token lifetime on login, `expiresInMins: 1`, so that expiry happens while you are testing. Decide what your app does when the token expires mid session, and make sure the user does not lose their place or get a blank screen.

Stock list. Paginated. The catalogue has 194 items. Show enough per item to be useful. Include a category filter, a sort control, and a search box.

Item detail. A dedicated route, `/items/:id` or similar. Someone must be able to paste that URL into chat and have a colleague open the same item.

Stock correction. From the item detail, let a user set a new stock count and save it. `PUT /products/{id}`. Decide how the UI behaves between the click and the response, and what happens if the request fails.

Requirements that will be tested

These are written as user outcomes on purpose. How you achieve them is your call.

1. Typing in the search box must never leave the user looking at results for a query they have already replaced, even on a slow connection. Verify this yourself using the `delay` parameter.
2. Changing the category filter or the sort order must not strand the user on an empty page.
3. Reloading the browser must put the user back where they were: same search, same filter, same sort, same page. So must opening a copied URL on another machine.
4. Every screen that loads data has a loading state, an empty state and an error state. The error state must offer the user a way to recover. Test the error path against `/http/500`.
5. The whole app is usable with a keyboard alone, and readable at 360px wide.

Code quality and tooling

We read your repository the way we would read a colleague's. Set it up so that the machine enforces the boring things and the review can be about the interesting ones.

- **A formatter, configured and committed.** Prettier, or the equivalent for your stack. Include a `format: check` script that fails when files are unformatted.
- **A linter, configured and committed.** ESLint or your framework's equivalent, with a ruleset you have actually chosen rather than an untouched default. Any rule you disable needs a one-line comment saying why.
- **Conventional Commits, enforced by commitlint.** Wire it to a commit-msg hook with husky, lefthook or equivalent, so it runs locally and not only in CI.
- **An `.editorconfig` file**, committed, so formatting stays stable across editors.
- **Formatting, linting and commit checks run in your pipeline** and must be able to fail it. See Section 3.
- **Keep reformatting out of feature commits.** A diff that is mostly whitespace is hard to review, and we will be reviewing it.

Constraints

- Structure the code sensibly. Not everything in one file.
- Include tests on the logic that is easy to get wrong. A placeholder test file scores zero. Two or three meaningful tests beat twenty shallow ones.
- Note in your README any limitation of the mock API that affected your implementation, and what you did about it.

Optional, if you have time

Bulk correction of several items at once. An offline or reconnect indicator. Virtualised scrolling of the full 194 items. Do not attempt these at the cost of the required behaviour.

SAVANNAH INFORMATICS
CONFIDENTIAL

DEPLOYMENT & CI/CD

Deploy to any provider: Vercel, Netlify, Render, Fly.io, Cloudflare Pages, GitHub Pages, AWS, GCP. The app must be reachable at a public URL at the moment you submit, and still reachable when we review it.

CI/CD pipeline

Set up a pipeline with any CI tool that:

- runs your formatter check, your linter and your commit message check on every pull request,
- runs your test suite on every pull request,
- fails the pull request when any of those checks fail,
- deploys automatically when a pull request is merged into your chosen branch.

What to include in your README

- The public URL of your deployed application.
- The branch that triggers a deployment.
- A few lines on what the pipeline does, and which checks can block a merge.

SECTION 4

AI REFLECTION

Answer these in your README or a separate file. Bullets are fine. Short and specific beats long and polished.

1. What did you use AI for across the four sections? Answer per section, not in general.
2. Which tools did you use? Include any spec-driven development or agent workflow framework, for example Superpowers, GSD, Spec Kit, OpenSpec or BMAD, and describe how the workflow actually ran for you. If you did not use one, tell us how you structured the work instead.
3. Give one example where an AI suggestion improved your work. What did you prompt it with?
4. Give one example where AI output was wrong, incomplete or subtly bad, and how you caught it.
5. Name two decisions you made without AI, and why you trusted your own judgment there.
6. Point us at one part of your codebase you would struggle to defend, and tell us why.

NOTE: Honesty here counts for more than the answer itself. Question 6 in particular is not a trap. Knowing where your own understanding is thin is a senior habit, and we would rather you told us than hoped we would not notice.

WHAT HAPPENS AFTER YOU SUBMIT

If your submission moves forward, we run a 75 minute live session. It has four parts:

5. You walk us through your own code. We will pick sections and ask why they are the way they are, and what breaks if we remove them.
6. We share a short component with you and ask you to review it.
7. We describe a production incident and ask you to work out what is happening.
8. We ask you to sketch or start a small extension to what you built.

You do not need to prepare anything. Bring the machine you built on.

Savannah Informatics does not discriminate on the basis of race, age, colour, religion, national origin or ancestry, sex, gender, disability, veteran status, genetic information, sexual orientation, gender identity, or expression.